SOLAR▸SWARM

Integration Guide
Martin Pham | Perth, AU

Voice AI Guide

Overview

The voice AI stack enables automated outbound calls to hot leads and inbound call handling for solar company clients. It uses **Retell AI** as the telephony platform and **ElevenLabs** for premium voice quality, with **GPT-4o** as the intelligence layer.

Files:

- `voice/call_handler.py` — Flask webhook server (port 5002)
- `voice/retell_client.py` — Retell AI API client
- `voice/post_call.py` — Post-call transcript processor
- `voice/prompt_templates.py` — System prompt templates
- `voice/call_functions.py` — Function definitions for GPT-4o tool calls

Retell AI Integration

How Calls Are Initiated

Inbound calls:

1. Customer calls the solar company's Retell-managed phone number
2. Retell sends `POST /voice/call-started` to initialise call context
3. For each agent turn, Retell sends `POST /voice/response` with the transcript so far
4. Our server calls GPT-4o with the full system prompt and returns the response
5. When the call ends, Retell sends `POST /voice/post-call` with the full transcript

Outbound calls (hot leads):

1. `qualification_agent` qualifies a lead with score ≥ 7 and phone present
2. Calls `retell_client.create_outbound_call(from_number, to_number, agent_id, metadata)`
3. Retell initiates the call and the same webhook flow applies

Call Flow Diagram

```
Customer ———————————— Retell AI Platform
    |                              |
    | Dials solar company number   |
    | ————————————————————————→ |
    |                              |
    |                              | POST /voice/call-started
    |                              | ————————————————————→ Solar Swarm
    |                              |                          |
    |                              |                          | Init call context
    |                              |                          | Log to call_logs
    |                              | ←———————————————— |
    |                              |
    | Speaks...                    |
    | ————————————————————→ |
    |                              | POST /voice/response
    |                              | (transcript so far)
    |                              | ————————————————————→ Solar Swarm
    |                              |                          |
    |                              |                          | Build system prompt
    |                              |                          | Call GPT-4o
    |                              |                          | Execute function calls
    |                              | ←———————————————— |
    |                              |   {content: "...", end_call: false}
    |                              |
    | ←———————————————— |  (AI speaks response)
    |                              |
    | [Call ends]                  |
    |                              | POST /voice/post-call
    |                              | (full transcript + recording)
    |                              | ————————————————————→ Solar Swarm
    |                              |                          | post_call.py
    |                              |                          | Extract → Score → GHL → Slack
```

Setting Up a Client's Voice Agent

Use `retell_client.setup_client_voice_agent()` to do the full setup in one call:

```
from voice.retell_client import setup_client_voice_agent

result = setup_client_voice_agent(
    client_id="sunpower_perth",
    company_name="SunPower Perth",
    phone_number="+61892345678",       # Must be in E.164 format
    webhook_base_url="https://your-server.com",
    elevenlabs_voice_id="your-voice-id",  # Optional
)
# result: {success, agent_id, phone, llm_url, phone_linked}
```

This creates the Retell agent, links the phone number, and saves the `agent_id` to the company profile in the database.

ElevenLabs Voice Selection

ElevenLabs provides higher-quality, more natural-sounding Australian voices than Retell's native TTS.

Default voice: `11labs-Adrian` (configured as `RETELL_DEFAULT_VOICE_ID`)

To use a custom ElevenLabs voice:

1. Create a voice in ElevenLabs dashboard or clone a client's voice
2. Copy the voice ID
3. Pass it to `setup_client_voice_agent(elevenlabs_voice_id="...")`
4. Or set `ELEVENLABS_DEFAULT_VOICE` in `.env` for all new agents

ElevenLabs character cost is tracked per call via `tracking/cost_tracker.py`.

Call Handler Routes

File: `voice/call_handler.py`, served on `PORT_VOICE_WEBHOOK` (default 5002)

METHOD	PATH	DESCRIPTION
GET	`/voice/health`	Health check
POST	`/voice/call-started`	Retell call initialisation
POST	`/voice/response`	Main LLM endpoint — Retell sends transcript, we return next response
POST	`/voice/post-call`	Post-call transcript + recording processing
POST	`/voice/elevenlabs/response`	ElevenLabs alternative to `/voice/response`
POST	`/webhook/email-received`	GHL email forwarding (registered at runtime in `main.py`)

`/voice/call-started` Payload

```
{
  "call_id": "call_abc123",
  "from_number": "+61412345678",
  "to_number": "+61892345678",
  "agent_id": "agent_xyz"
}
```

Initialises `_call_contexts[call_id]` with client_id, phones, and empty lead_data.

`/voice/response` Payload (Retell Custom LLM format)

```
{
  "call_id": "call_abc123",
  "interaction_type": "response_required",
  "response_id": 5,
  "transcript": [
    {"role": "agent", "content": "Hi! Thanks for calling SunPower Perth."},
    {"role": "user", "content": "Hi, I'm interested in solar panels."}
  ]
}
```

Response:

```
{
  "response_id": 5,
  "content": "Great! Let me ask you a few quick questions...",
  "content_complete": true,
  "end_call": false
}
```

`/voice/post-call` Payload

Retell sends the full call data after hang-up:

```
{
  "call_id": "call_abc123",
  "transcript": [...],
  "duration_ms": 180000,
  "recording_url": "https://retell.com/recordings/...",
  "agent_id": "agent_xyz"
}
```

Post-Call Transcript Processing

File: `voice/post_call.py` — `process_post_call(webhook_data, call_ctx)`

Nine steps executed after every call ends:

1. **Extract** — GPT-4o parses the full transcript into structured lead data (name, email, suburb, state, homeowner_status, monthly_bill, roof_type, roof_age, outcome, sentiment, key_objections, follow_up_notes, call_summary)
1. **Merge** — Extracted data merged with in-call context accumulated during the call
1. **Upsert lead** — Creates or updates the `leads` table record
1. **Score** — If not already scored, calls `qualification_agent.qualify()` on the extracted data
1. **Update GHL** — Updates custom fields, adds tags based on outcome, syncs recording URL
1. **Create task** — If `follow_up_required=True`, creates a dated task in GHL for the sales team

1. **Update call_logs** — Stamps duration, recording URL, outcome, score on the `call_logs` record
1. **Slack notification** — Posts call outcome with score emoji and summary
1. **Cold ledger** — Writes `VOICE_CALL_COMPLETE` event

GHL tags applied by outcome:

OUTCOME	TAGS ADDED
`booked_assessment`	assessment-booked, voice-ai-qualified
`callback_requested`	callback-requested, voice-ai-lead
`not_interested`	not-interested-voice
`transferred`	transferred-to-human
Score ≥ 7	hot-lead
Battery interest detected	battery-interest
EV owner detected	ev-owner

Prompt Templates

File: `voice/prompt_templates.py`

The system supports multiple prompt templates for different call scenarios. The template is selected via `_build_system_prompt(client_id, call_id, template)`.

TEMPLATE NAME	USE CASE
`inbound_solar`	Default; inbound lead qualification
`outbound_cold`	Cold outreach to scraped prospects
`outbound_callback`	Hot lead callback (score ≥ 7)
`support`	Existing customer support

Template variables injected:

VARIABLE	SOURCE
`{company_name}`	`company_profiles` table or DEFAULT_CLIENT_ID
`{company_knowledge}`	`knowledge/company_kb.py` — products, pricing, rebates
`{today}`	Current date
`{call_id}`	From Retell

Voice Rules Enforced by System Prompt

- Keep responses under 3 sentences
- No bullet points — weave into natural speech
- Australian English (colour, neighbourhood, organise, mum)
- Never say "as an AI" unless directly asked
- Call `lookup_caller` at the very start of every call
- Call `qualify_and_score` once homeowner status, bill, and roof info are collected
- Call `end_call` when the conversation is naturally complete

How to Add a New Voice Prompt Template

1. Open `voice/prompt_templates.py`
2. Add a new key to the `TEMPLATES` dict with your prompt string
3. Include the same `{company_name}`, `{company_knowledge}`, `{today}`, `{call_id}` placeholders
4. Call `_build_system_prompt(client_id, call_id, template="your_new_template")` in the handler

Cost Tracking Per Call

Every call has its costs tracked in the `api_usage` table via `tracking/cost_tracker.py`:

Retell AI cost: Logged in `voice/call_handler.py` post-call handler

```
log_retell_call(
    call_id=call_id,
    duration_seconds=duration_s,
    client_id=ctx.get("client_id"),
    agent_id=data.get("agent_id"),
)
```

OpenAI tokens: Logged on every GPT-4o call inside `_call_llm()`:

```
log_openai(
    model=config.OPENAI_MODEL,
    prompt_tokens=usage.prompt_tokens,
    completion_tokens=usage.completion_tokens,
)
```

To see cost for a specific call:

```
curl "http://localhost:5000/costs?call_id=call_abc123"
```

Retell pricing (approximate): ~$0.02–$0.05 USD per minute depending on plan.

ElevenLabs pricing: Per character of generated speech; typical call uses ~500–2000 characters.

GPT-4o pricing: ~8–20 API calls per call; ~$0.01–$0.05 per call depending on length.